

NASA SPACE APPS CHALLENGE 2026 • OFFICIAL PROJECT DOSSIER

MISSIONFORGE

Interactive Space Mission Systems Engineering & Flight Simulation Game

Track: Space Mission Design
Game

Architecture: Client-Side WebGL &
React 18

Format: Executive Overview & Presentation
Guide

1. Executive Project Overview

MISSIONFORGE is a web-based, scientifically grounded interactive space engineering simulation designed to demystify interplanetary exploration. In typical space games, gameplay is restricted to arcade shooting or simplified orbital maneuvering. **MISSIONFORGE** shifts the paradigm by placing players in the shoes of a **NASA Mission Director & Chief Spacecraft Systems Engineer**.

Players formulate real aerospace mission architectures, assemble modular spacecraft subsystems, solve authentic engineering trade-offs (mass budgets, power deficits, communication latency, and launch vehicle lift capacities), undergo rigorous Flight Readiness Reviews, and navigate live flight anomalies during interplanetary transit.

Core Motto: *"Design. Decide. Launch."*

The game bridges the gap between academic astrophysics and gamified STEM engagement, allowing players to plan and fly full missions in a focused 3-to-5 minute interactive loop.

2. Code Languages, Libraries & Tech Stack

MISSIONFORGE was engineered using cutting-edge web technologies to achieve high-fidelity 3D graphics directly within modern browsers without requiring plugins or heavy downloads:

Domain	Code Language / Tool	Functionality & Strategic Benefit
Primary Languages	TypeScript / JavaScript (ES6+), HTML5, CSS3	100% type-safe calculations for mass, delta-v, and power budgets; modular components.
UI Framework	React 18	Declarative state management handling multi-stage screens, real-time recalculations, and telemetry updates.
Build Engine	Vite	Lightning-fast development bundling, immediate HMR, and ultra-optimized production assets.
3D Graphics & Physics	Three.js + React Three Fiber (@react-three/fiber)	Full WebGL rendering of the procedural 3D spacecraft, solar system celestial bodies, and launch vehicles at 60 FPS.
3D Ecosystem	@react-three/drei & Postprocessing	Orbital camera manipulation (OrbitControls), atmospheric skyboxes, bloom thruster glows, and depth-of-field.
Design & Styling	Tailwind CSS	NASA Mission Control HUD styling, glassmorphism telemetry panels, and custom dark sci-fi color tokens.
Motion & FX	Framer Motion & Canvas Confetti	Smooth UI view transitions, dial animations, and celebration particle physics upon mission success.
Iconography	Lucide React	Clean, scalable vector icons representing aerospace telemetry, sensors, and hardware.
Architecture	100% Client-Side Deterministic Engine	No backend database bottleneck; instant loading, zero latency, and complete offline capability.

3. Step-by-Step: How It Works & Gameplay Flow

The game takes the user through a realistic 10-phase aerospace mission pipeline:

Phase 01: Mission Brief Formulation

Select from authentic NASA profiles: *Lunar Polar Exploration* (Artemis water ice), *Mars Jezero Surface Science*, *Asteroid Survey & Sample* (Bennu/Psyche), *Earth Climate Observation*, or the sci-fi colony arc *Project New Eden*.

Phase 02: 3D Solar System Trajectory

Interactive orbital view showing real transit distances, surface gravity, cosmic radiation levels, vacuum thermal gradients, and communications latency.

Phase 03: Procedural 3D Spacecraft Hangar

Select modular chassis buses (Light, Standard, Heavy). The real-time 3D model immediately updates structural decks and radiation vaulting.

Phase 04: Scientific Payload Suite

Mount 8 real instruments (SAR Radar, Hyperspectral Spectrometer, Magnetometer, GPR, Radiation Detectors). Every sensor increases science points but drains power and adds mass.

Phase 05: Launch Vehicle Class

Match spacecraft mass against Light, Medium, Heavy, or Super Heavy rocket architectures. Over-capacity spacecraft trigger visual warning flags.

Phase 06: Subsystem Balancing

Balance electrical power (Solar vs Nuclear RTG), communications (Low-gain omni vs High-gain Cassegrain dish), and propulsion (Chemical vs Xenon Ion).

Phase 07: Trajectory Optimization

Choose between Hohmann-style Balanced Transfer, Direct Hyperbolic Fast Transfer, or Low-Energy Ballistic Capture.

Phase 08: Flight Readiness Review (FRR)

A strict 6-point automated audit of mass margins, power margins, budget, and delta-v. Critical failures block launch approval.

Phases 09 & 10: Launch, Interactive Minigames & Official NASA Flight Report

During flight, players experience interactive action minigames:

- **Ascent Launch Minigame:** Steer the rocket through atmospheric Max-Q, stage separation, and fairing jettison.
- **3D Cockpit Telemetry View:** Switch to the pilot/avionics perspective to monitor live heading, pitch, and engine telemetry.
- **In-Flight Anomalies:** Handle sudden solar particle events or micro-meteoroids with real-time tactical trade-offs.
- **Planet Lander Minigame:** Take manual throttle control to softly land on extraterrestrial terrain without crashing.
- **NASA Flight Debriefing Report:** Outputs an official deterministic evaluation report with a 0-100% mission score, lessons learned, and PDF export functionality.

4. Space Science Principles Learned Through Gameplay

- **Inverse-Square Law of Solar Radiation:** Players learn why outer solar system missions (e.g. Jupiter/Saturn/Deep Space) cannot rely on solar panels—solar flux diminishes with the square of the distance, requiring Radioisotope Thermoelectric Generators (RTGs).
- **Rocket Equation & Delta-V (ΔV):** Teaches why every extra kilogram of science payload demands exponentially more rocket propellant and structural cost.
- **Deep Space Communications Latency:** Demonstrates why Mars rovers require autonomous landing algorithms: at 14 to 22 light-minutes round-trip delay, manual joystick control from Earth is impossible.
- **Planetary Environments:** Imparts genuine scientific data on lunar water ice trapped inside permanently shadowed craters, Martian perchlorates, and asteroid microgravity orbital perturbations.

5. Complete Word-for-Word English Presentation Script (3–4 Minutes)

Use this exact script for live pitches, demonstrations, and competition reviews:

[0:00 – 0:45] PHASE 1: THE HOOK & INTRODUCTION

"Honorable judges, teachers, and space enthusiasts — good morning/afternoon.

*When we think of space exploration, we immediately envision the thunderous roar of rockets and astronauts landing on foreign worlds. But behind every single launch lies the real hero of space exploration: **Systems Engineering**.*

How does a NASA team design a spacecraft that can survive temperatures of minus 200 degrees Celsius, navigate across hundreds of millions of kilometers, operate autonomously during a 20-minute communication blackout, and still stay within strict congressional budgets and rocket payload limits?

*To bring these real-world aerospace challenges to life, we created **MISSIONFORGE: Space Mission Design Game**."*

[0:45 – 1:30] PHASE 2: TECH STACK & ARCHITECTURE

"To make this simulation accessible to anyone directly inside a web browser without requiring hefty downloads, we engineered a modern, high-performance web architecture:

- The core engine is built with **React 18 and TypeScript**, guaranteeing modular design and rock-solid type safety.*
- For real-time 3D visualization, we utilized **Three.js and React Three Fiber**, allowing players to inspect full 3D orbital trajectories, solar systems, and spacecraft geometries at a silky-smooth 60 frames per second.*
- The user interface is powered by **Tailwind CSS and Framer Motion**, styled after NASA Mission Control Center consoles with glassmorphism and real-time Heads-Up Displays.*
- Best of all, MISSIONFORGE runs **100% on the client side**. The physics engine and engineering math execute instantly with zero server lag."*

[1:30 – 2:30] PHASE 3: WHAT WE LEARN ABOUT SPACE

"MISSIONFORGE transforms dry textbook astrophysics into intuitive, interactive discoveries:

First, **Destinations & Environments**: Players can embark on real mission profiles—such as prospecting water ice in the permanently shadowed craters of the Lunar South Pole for the Artemis program, or hunting for ancient microbial fossils in Mars' Jezero Crater.

Second, **The Crucial Trade-offs**: In space, you cannot simply add everything you want. If you install heavy synthetic aperture radars and optical spectrometers, your spacecraft's weight skyrockets. That demands a bigger, costlier rocket, which drains your budget.

Third, **Real Planetary Physics**: Players physically see the **Inverse-Square Law** in action—as you travel further from the Sun, your solar panels produce less power, forcing you to choose Nuclear Stirling RTGs instead. They also experience communication latency, learning why Mars rovers must make autonomous landing decisions because Earth is 14 light-minutes away."

[2:30 – 3:15] PHASE 4: HANDS-ON MINIGAMES & GAMEPLAY DEMO

"Our project isn't just a static calculator — it's an engaging, multi-phase game:

- Players assemble their probe in the **3D Hangar**, watching solar panels and sensors snap onto the chassis.
- Before launch, our automated **Flight Readiness Review** audits all 6 critical engineering criteria.
- During launch, players pilot the **Ascent Minigame**, managing pitch angles, staging, and fairing deployment.
- During transit, live **Flight Anomalies** trigger—forcing the player to choose between conserving battery power or maintaining high science bandwidth.
- And during arrival, players take manual control in the **3D Planet Lander Minigame**, delicately firing descent engines to touch down safely on extraterrestrial soil."

[3:15 – 3:45] PHASE 5: CONCLUSION & FLIGHT REPORT

"At the conclusion of each voyage, the game generates an official **NASA Flight Evaluation Report**, scoring the mission across scientific return, budget efficiency, and telemetry integrity. Players can even export and print this debriefing.

In conclusion, **MISSIONFORGE** bridges the gap between pure gaming and rigorous aerospace education. It empowers the next generation of students and explorers to design, decide, and launch.

Thank you very much, and we are now open to any questions!"

6. Quick Q&A Defense for Judges

Q: How realistic is the physics?

A: The core mathematical models are grounded in aerospace facts: launcher lift capacities match actual rocket classes; solar flux models follow the $1/r^2$ inverse-square law; communications delays reflect actual light travel times; and delta-v balances chemical vs. ion specific impulse.

Q: What makes MISSIONFORGE different from Kerbal Space Program?

*A: KSP requires hours of steep orbital physics piloting. MISSIONFORGE targets the critical **Systems Engineering & Decision Trade-offs** of mission planning, making high-level aerospace concepts accessible to everyone in 3–5 minutes.*